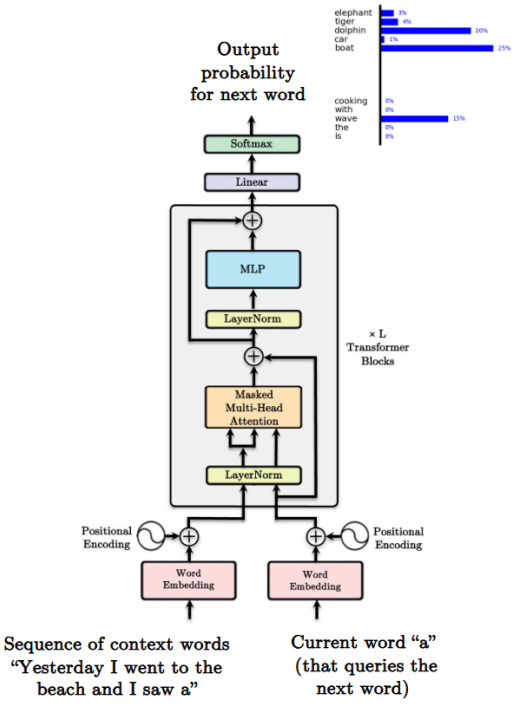
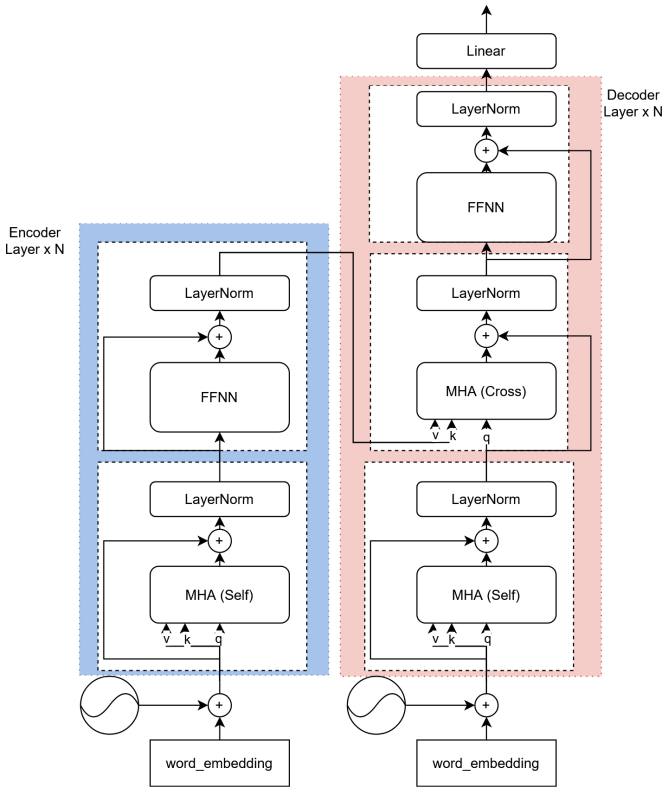
Transformers

1. Motivation [8]:
    1. Different senses, but same embedding: word2vec, GloVe, or other static embeddings assign exactly the same embedding to each of the word 'basin' in 'Amazon basin' vs 'she filled the basin with water' or the word 'bank' in 'river bank' vs 'bank' despite their different senses.
    2. we want to assign different, **contextualized embeddings** based on the surrounding context.
2. Transformers can [2]:
    1. process an unordered set of tokens/vectors
        1. Positional encoding adds spatial information.
    2. self-attention is like clustering and replacing vectors with centers, except
        1. Multiple, complex, learnable similarity functions.
        2. No need to preset number of clusters.
    3. Vectors are iteratively aggregated and transformed.
    4. Vectors can start as purely local information (e.g. 16x16 patch).
3. What kind of word embedding is used in the original transformer?
    1. Recall example of word embeddings such as one-hot, word2vec, GloVe, etc.
    2. There are *two options* for dealing with the Pytorch nn.Embedding weight matrix: [3]
        1. Initialize it with pre-trained embeddings and keep it fixed, in which case it's really just a lookup table.
        2. Initialize it randomly, or with pre-trained embeddings, but keep it trainable. In that case the word representations will get refined and modified throughout training because the weight matrix will get refined and modified throughout training. The transformer uses a random initialization of the weight matrix and refines these weights during training - i.e. it learns its own word embeddings. Note: Recall in backpropagation we can get $\frac{\partial L}{\partial \vec{x}}$.
4. Transformer encoder[6]
    1. The encoder in seq2seq model can replace recurrence through a series of multi-head self-attention blocks.
    2. But this ignores relative position.
        1. A context word one word away is different from one 10 words away.
        2. The attention framework does not take distance into consideration. This can be an issue, for example when, there are multiple duplicated words in a sentence.
    3. Add positional encoding to them to capture the relative distance from one another.
    4. Positional encoding cannot be any random function: A sequence of vectors $P_0, \ldots, P_N$ to encode position
        1. Every vector is unique and uniquely represents time
        2. Relationship between P_t and $P_{t+\tau}$ only depends on the distance between them $P_{t+\tau} = M_\tau P_t$
        3. The linear relationship between P_t and $P_{t+\tau}$ enables the net to lean shift-invariant "gap" dependent relationships
        4. $P_{t+\tau} = M_\tau P_t$
        5. $M_\tau = diag\left(\begin{bmatrix} \cos\omega_l\tau & \sin\omega_l\tau \\ -\sin\omega_l\tau & \cos\omega_l\tau \end{bmatrix}, l = 1, \ldots, d/2\right)$
        6. A vector of sines and cosines of a harmonic series of frequencies
            1. Every $2l$-th component of P_t is $\sin\omega_l t$
            2. Every $2l+1$-th component of P_t is $\cos\omega_l t$
            3. Never repeats
            4. Has the linearity property required
    5. BERT uses the encoder side.
5. Transformer decoder [4], [5]
    1. in NMT the partially decoded sentence is used as query in the first multi-head attention block.
    2. the need for decoder
        1. The decoder is used when the output of the model is a sequence.
        2. If the output of the model is single value, e.g. for a classification task or a single-value regression task, the encoder would suffice.
        3. When predicting multiple values of a time series, a decoder should probably be used that let to condition not only on the input, but also on the partially generated output values.
    3. Decoder = Conditional generator
        1. In math: $P(y_t|x_1, \ldots, x_T, y_1, \ldots, y_{t-1})$

    2. where $x_1, \ldots, x_T$ is the input sequence, $y_1, \ldots, y_{t-1}$ are all the past output items.
    3. Each item in the output sequence must be conditioned on:
        1. The entire input sequence.
        2. All the past output items.
    4. In deep learning the decoder must have access to:
        1. Some kind of encoding of the entire input sequence.
        2. The past states of the decoder.
4. Very similar to the encoder.
5. Masked MHA block to hide future output values during training.
6. The query from the decoder is used with the keys/values from the encoder.
7. Final output probabilities are computed using a projection layer followed by a softmax.
8. GPT only uses the decoder.
9. Decoder self-attention and masked attention:
    1. Decoders can be parallelized during training (only).
    2. Feed the whole output sequence (outputs) in all at once.
    3. Need to ensure model doesn't cheat.
    4. Alter the attention weights to be 0 (set input to softmax to -inf) for all times $t' > t$.
    5. Ensure autoregressive property.
10. Cross-attention
    1. During decoding, the query comes from the outputs, keys and values come from the encoder.
    2. Decoder input "pays" attention to the encoder outputs.
11. Feedforward layers allow for high dimensional computations (nonlinearity).
12. Simply there to allow the model to capture more information.

6. self-attention computed for an T length input requires the computation of $T \times T$ attention weight matrix for each head.
7. Masked self-attention is required for all layers of the decoder
8. We can combine recurrent layers with self-attention layers.
9. Positional encodings are the same in the encoder and decoder, the only thing different is the mask.
10. Transformers are residual machines.
    1. Add & Norm: out = LayerNorm(x + Sublayer(x)),
    2. Sublayer(x) is whatever layer is below the Add & Norm.





11. Code: transformer_layers.py

```
import torch
import torch.nn as nn
from torch.nn import functional as F
import math

"""
This file defines layer types that are commonly used for transformers.
"""


class PositionalEncoding(nn.Module):
    """
    Encodes information about the positions of the tokens in the sequence. In
    this case, the layer has no learnable parameters, since it is a simple
    function of sines and cosines.
    """

    def __init__(self, embed_dim, dropout=0.1, max_len=5000):
        """
        Construct the PositionalEncoding layer.

        Inputs:
         - embed_dim: the size of the embed dimension
         - dropout: the dropout value
         - max_len: the maximum possible length of the incoming sequence
        """
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)
        assert embed_dim % 2 == 0
        # Create an array with a "batch dimension" of 1 (which will broadcast
        # across all examples in the batch).
        pe = torch.zeros(1, max_len, embed_dim)
        ############################################################################
        # TODO: Construct the positional encoding array as described in            #
        # Transformer_Captioning.ipynb.  The goal is for each row to alternate     #
        # sine and cosine, and have exponents of 0, 0, 2, 2, 4, 4, etc. up to      #
        # embed_dim. Of course this exact specification is somewhat arbitrary, but #
        # this is what the autograder is expecting. For reference, our solution is #
        # less than 5 lines of code.                                               #
        ############################################################################
        # Create array with all positions [0, 1, 2, ..., (max_len-1)]
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        # Compute all possible i/(10000^(2j/embed_dim)) values
        dimensions = torch.arange(0, embed_dim, 2).float()
        div_term = 1 / torch.pow(10000, (dimensions / embed_dim))
        # Compute sinusoidal functions
        pe[:, :, 0::2] = torch.sin(position * div_term)
        pe[:, :, 1::2] = torch.cos(position * div_term)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # Make sure the positional encodings will be saved with the model
```

```
        # parameters (mostly for completeness).
        self.register_buffer("pe", pe)

    def forward(self, x):
        """
        Element-wise add positional embeddings to the input sequence.

        Inputs:
         - x: the sequence fed to the positional encoder model, of shape
              (N, S, D), where N is the batch size, S is the sequence length and
              D is embed dim
        Returns:
         - output: the input sequence + positional encodings, of shape (N, S, D)
        """
        N, S, D = x.shape
        # Create a placeholder, to be overwritten by your code below.
        output = torch.empty((N, S, D))
        ############################################################################
        # TODO: Index into your array of positional encodings, and add the         #
        # appropriate ones to the input sequence. Don't forget to apply dropout    #
        # afterward. This should only take a few lines of code.                    #
        ############################################################################
        pos_enc = self.pe[:, : x.shape[1], :]
        output = x + pos_enc
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################
        return self.dropout(output)


class MultiHeadAttention(nn.Module):
    """
    A model layer which implements a simplified version of masked attention, as
    introduced by "Attention Is All You Need" (https://arxiv.org/abs/1706.03762).

    Usage:
      attn = MultiHeadAttention(embed_dim, num_heads=2)

      # self-attention
      data = torch.randn(batch_size, sequence_length, embed_dim)
      self_attn_output = attn(query=data, key=data, value=data)

      # attention using two inputs
      other_data = torch.randn(batch_size, sequence_length, embed_dim)
      attn_output = attn(query=data, key=other_data, value=other_data)
    """

    def __init__(self, embed_dim, num_heads, dropout=0.1):
        """
        Construct a new MultiHeadAttention layer.

        Inputs:
         - embed_dim: Dimension of the token embedding
         - num_heads: Number of attention heads
         - dropout: Dropout probability
        """
        super().__init__()
        assert embed_dim % num_heads == 0

        # We will initialize these layers for you, since swapping the ordering
        # would affect the random number generation (and therefore your exact
        # outputs relative to the autograder). Note that the layers use a bias
        # term, but this isn't strictly necessary (and varies by
        # implementation).
        self.key = nn.Linear(embed_dim, embed_dim)
        self.query = nn.Linear(embed_dim, embed_dim)
        self.value = nn.Linear(embed_dim, embed_dim)
        self.proj = nn.Linear(embed_dim, embed_dim)

        self.attn_drop = nn.Dropout(dropout)

        self.n_head = num_heads
        self.emd_dim = embed_dim
        self.head_dim = self.emd_dim // self.n_head

    def forward(self, query, key, value, attn_mask=None):
        """
        Calculate the masked attention output for the provided data, computing
        all attention heads in parallel.

        In the shape definitions below, N is the batch size, S is the source
        sequence length, T is the target sequence length, and E is the embedding
        dimension.

        Inputs:
        - query: Input data to be used as the query, of shape (N, S, E)
        - key: Input data to be used as the key, of shape (N, T, E)
        - value: Input data to be used as the value, of shape (N, T, E)
        - attn_mask: Array of shape (S, T) where mask[i,j] == 0 indicates token
          i in the source should not influence token j in the target.
```

```
        Returns:
        - output: Tensor of shape (N, S, E) giving the weighted combination of
          data in value according to the attention weights calculated using key
          and query.
        """
        N, S, E = query.shape
        N, T, E = value.shape
        # Create a placeholder, to be overwritten by your code below.
        output = torch.empty((N, S, E))
        ############################################################################
        # TODO: Implement multiheaded attention using the equations given in       #
        # Transformer_Captioning.ipynb.                                            #
        # A few hints:                                                             #
        #  1) You'll want to split your shape from (N, T, E) into (N, T, H, E/H),  #
        #     where H is the number of heads.                                      #
        #  2) The function torch.matmul allows you to do a batched matrix multiply.#
        #     For example, you can do (N, H, T, E/H) by (N, H, E/H, T) to yield a  #
        #     shape (N, H, T, T). For more examples, see                           #
        #     https://pytorch.org/docs/stable/generated/torch.matmul.html          #
        #  3) For applying attn_mask, think how the scores should be modified to   #
        #     prevent a value from influencing output. Specifically, the PyTorch   #
        #     function masked_fill may come in handy.                              #
        ############################################################################
        from einops import rearrange

        p_q, p_k, p_v = self.query(query), self.key(key), self.value(value)
        r_q = rearrange(p_q, "n s (h d) -> n h s d", h=self.n_head)
        r_k = rearrange(p_k, "n t (h d) -> n h t d", h=self.n_head)
        r_v = rearrange(p_v, "n t (h d) -> n h t d", h=self.n_head)

        scores = (
            torch.einsum("n h s d, n h t d -> n h s t", r_q, r_k) / self.head_dim**0.5
        )
        if attn_mask is not None:
            print("scores.shape", scores.shape)
            print("attn_mask.shape", attn_mask.shape)
            scores = torch.masked_fill(scores, ~attn_mask, -torch.inf)

        attn = torch.nn.functional.softmax(scores, dim=-1)
        attn = self.attn_drop(attn)
        result = torch.einsum("n h s t, n h t d -> n h s d", attn, r_v)
        r_result = rearrange(result, "n h s d -> n s (h d)")
        output = self.proj(r_result)

        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################
        return output


class FeedForwardNetwork(nn.Module):
    def __init__(self, embed_dim, ffn_dim, dropout=0.1):
        """
        Simple two-layer feed-forward network with dropout and ReLU activation.

        Inputs:
         - embed_dim: Dimension of input and output embeddings
         - ffn_dim: Hidden dimension in the feedforward network
         - dropout: Dropout probability
        """
        super().__init__()
        self.fc1 = nn.Linear(embed_dim, ffn_dim)
        self.gelu = nn.GELU()
        self.dropout = nn.Dropout(dropout)
        self.fc2 = nn.Linear(ffn_dim, embed_dim)

    def forward(self, x):
        """
        Forward pass for the feedforward network.

        Inputs:
        - x: Input tensor of shape (N, T, D)

        Returns:
        - out: Output tensor of the same shape as input
        """
        out = torch.empty_like(x)

        out = self.fc1(x)
        out = self.gelu(out)
        out = self.dropout(out)
        out = self.fc2(out)

        return out


class TransformerDecoderLayer(nn.Module):
    """
    A single layer of a Transformer decoder, to be used with TransformerDecoder.
```

```
    """

    def __init__(self, input_dim, num_heads, dim_feedforward=2048, dropout=0.1):
        """
        Construct a TransformerDecoderLayer instance.

        Inputs:
         - input_dim: Number of expected features in the input.
         - num_heads: Number of attention heads
         - dim_feedforward: Dimension of the feedforward network model.
         - dropout: The dropout value.
        """
        super().__init__()
        self.self_attn = MultiHeadAttention(input_dim, num_heads, dropout)
        self.cross_attn = MultiHeadAttention(input_dim, num_heads, dropout)
        self.ffn = FeedForwardNetwork(input_dim, dim_feedforward, dropout)

        self.norm_self = nn.LayerNorm(input_dim)
        self.norm_cross = nn.LayerNorm(input_dim)
        self.norm_ffn = nn.LayerNorm(input_dim)

        self.dropout_self = nn.Dropout(dropout)
        self.dropout_cross = nn.Dropout(dropout)
        self.dropout_ffn = nn.Dropout(dropout)

    def forward(self, tgt, memory, tgt_mask=None):
        """
        Pass the inputs (and mask) through the decoder layer.

        Inputs:
        - tgt: target, the sequence to the decoder layer, of shape (N, T, W)
        - memory: the sequence from the last layer of the encoder, of shape (N, T, W)
        - tgt_mask: target mask, the parts of the target sequence to mask, of shape (T, T)

        Returns:
        - out: the Transformer features, of shape (N, T, W)
        """

        # Self-attention block (reference implementation)
        # shortcut = tgt
        # tgt = self.self_attn(query=tgt, key=tgt, value=tgt, attn_mask=tgt_mask)
        # tgt = self.dropout_self(tgt)
        # tgt = tgt + shortcut
        # tgt = self.norm_self(tgt)

        out1 = self.self_attn(query=tgt, key=tgt, value=tgt, attn_mask=tgt_mask)
        out1 = self.dropout_self(out1)
        out1 = self.norm_self(out1 + tgt)

        out2 = self.cross_attn(out1, memory, memory)
        out2 = self.dropout_cross(out2)
        out2 = self.norm_cross(out2 + out1)

        out3 = self.ffn(out2)
        out3 = self.dropout_ffn(out3)
        out3 = self.norm_ffn(out3 + out2)

        ##############################################################################
        # TODO: Complete the decoder layer by implementing the remaining two       #
        # sublayers: (1) the cross-attention block using the encoder output as     #
        # memory, and (2) the feedforward block. Each block should follow the      #
        # same structure as self-attention implemented just above.                 #
        ##############################################################################

        ##############################################################################
        #                               END OF YOUR CODE                             #
        ##############################################################################

        return out3


class PatchEmbedding(nn.Module):
    """
    A layer that splits an image into patches and projects each patch to an embedding vector.
    Used as the input layer of a Vision Transformer (ViT).

    Inputs:
    - img_size: Integer representing the height/width of input image (assumes square image).
    - patch_size: Integer representing height/width of each patch (square patch).
    - in_channels: Number of input image channels (e.g., 3 for RGB).
    - embed_dim: Dimension of the linear embedding space.
    """

    def __init__(self, img_size, patch_size, in_channels=3, embed_dim=128):
        super().__init__()

        self.img_size = img_size
        self.patch_size = patch_size
        self.in_channels = in_channels
        self.embed_dim = embed_dim
```

```
        assert (
            img_size % patch_size == 0
        ), "Image dimensions must be divisible by the patch size."

        self.num_patches = (img_size // patch_size) ** 2
        self.patch_dim = patch_size * patch_size * in_channels

        # Linear projection of flattened patches to the embedding dimension
        self.proj = nn.Linear(self.patch_dim, embed_dim)
        # from einops.layers.torch import Rearrange
        # self.to_patch_embedding = Rearrange('n c (h p1) (w p2) -> n (h w) (p1 p2 c)', p1=self.patch_size,
p2=self.patch_size)

    def forward(self, x):
        """
        Forward pass for patch embedding.

        Inputs:
        - x: Input image tensor of shape (N, C, H, W)

        Returns:
        - out: Patch embeddings with shape (N, num_patches, embed_dim)
        """
        N, C, H, W = x.shape
        assert (
            H == self.img_size and W == self.img_size
        ), f"Expected image size ({self.img_size}, {self.img_size}), but got ({H}, {W})"
        out = torch.zeros(N, self.embed_dim)

        ###########################################################################
        # TODO: Divide the image into non-overlapping patches of shape            #
        # (C x patch_size x patch_size), and rearrange them into a tensor of      #
        # shape (N, num_patches, patch_dim). Do not use a for-loop.               #
        # Instead, you may find torch.reshape and torch.permute helpful for this  #
        # step. Once the patches are flattened, embed them into latent vectors    #
        # using the projection layer.                                             #
        ###########################################################################
        from einops import rearrange

        # h = H / p1, w = W / p2
        # (h*w) = num_patches, (p1*p2*C) = patch_dim

        # lucidrains version
        # out = rearrange(x, 'N C (h p1) (w p2) -> N (h w) (p1 p2 C)', p1=self.patch_size, p2=self.patch_size)

        out = rearrange(
            x,
            "N C (h p1) (w p2) -> N (h w) (C p1 p2)",
            p1=self.patch_size,
            p2=self.patch_size,
        )
        # num_patch = self.img_size // self.patch_size
        # out = x.reshape(N, C, num_patch, self.patch_size, \
        #                     num_patch, self.patch_size).permute(0, 2, 4, 1, 3, 5)
        # out = out.reshape(N, self.num_patches, self.patch_dim)

        out = self.proj(out)

        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
        return out


class TransformerEncoderLayer(nn.Module):
    """
    A single layer of a Transformer encoder, to be used with TransformerEncoder.
    """

    def __init__(self, input_dim, num_heads, dim_feedforward=2048, dropout=0.1):
        """
        Construct a TransformerEncoderLayer instance.

        Inputs:
         - input_dim: Number of expected features in the input.
         - num_heads: Number of attention heads.
         - dim_feedforward: Dimension of the feedforward network model.
         - dropout: The dropout value.
        """
        super().__init__()
        self.self_attn = MultiHeadAttention(input_dim, num_heads, dropout)
        self.ffn = FeedForwardNetwork(input_dim, dim_feedforward, dropout)

        self.norm_self = nn.LayerNorm(input_dim)
        self.norm_ffn = nn.LayerNorm(input_dim)

        self.dropout_self = nn.Dropout(dropout)
        self.dropout_ffn = nn.Dropout(dropout)
```

```
    def forward(self, src, src_mask=None):
        """
        Pass the inputs (and mask) through the encoder layer.

        Inputs:
        - src: the sequence to the encoder layer, of shape (N, S, D)
        - src_mask: the parts of the source sequence to mask, of shape (S, S)

        Returns:
        - out: the Transformer features, of shape (N, S, D)
        """
        ############################################################################
        # TODO: Implement the encoder layer by applying self-attention followed    #
        # by a feedforward block. This code will be very similar to decoder layer. #
        ############################################################################

        out1 = self.self_attn(src, src, src, src_mask)
        out1 = self.dropout_self(out1)
        out1 = self.norm_self(out1 + src)

        out2 = self.ffn(out1)
        out2 = self.dropout_ffn(out2)
        out2 = self.norm_ffn(out1 + out2)

        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################
        return out2
```

References:

1. Formal Algorithms for Transformers, Mary Phuong, Marcus Hutter.
2. CS598, Derek Hoeim, UIUC.
3. The Transformer: Attention Is All You Need
4. Role of decoder in transformer
5. Attention Networks, CMU 11-785 S21, Recitation 9.
6. seq2seq models: Attention models. CMU 11-785, F22 Lecture 17.
7. Xavier Bresson, Lecture 7, Attention Neural Networks.
8. Natural Language Processing, UofU.